

Design Card — my_switch

NetworkArena A1 · Link-state routing switch program · submitted with my_switch.wasm

The problem

Every switch runs the same program with no view of the global topology — it only ever talks to its direct neighbors. The program has to solve two things on top of the given neighbor discovery and liveness scaffolding: **propagation** (get every customer prefix into every switch's forwarding table) and **recovery** (notice a dead link and re-route around it inside the recovery budget).

Why link-state, not distance-vector or flood-and-cache

I picked **link-state** (flood link descriptions, compute shortest paths locally with Dijkstra) over distance-vector or naive flood-and-cache. My reasoning: distance-vector is the least code, but its classic failure mode — slow convergence / count-to-infinity when a route disappears — is exactly what C4 (recovery) grades. Flood-and-cache propagates prefixes fine but gives every switch only local reachability info, not a topology map, so there's no way to compute a good next-hop or to reason about paths going stale when a link far away dies. Link-state costs more code (I have to run Dijkstra inside a fuel-limited handler) but its failure behavior is the most predictable: every switch recomputes from a full picture instead of propagating a possibly-stale distance guess.

How it works

- 1. Neighbor discovery & liveness (given).** HELLO every 50 ms per port; a neighbor is declared down after 150 ms of silence (≈ 3 missed HELLOs — tolerates one drop without a false positive, while staying well under the 1000 ms recovery budget).
- 2. What gets flooded (the LSP).** Each switch floods its own switch id, a sequence number, its list of (up-neighbor, cost) pairs, and the customer prefixes it directly serves. My intuition: announcing links alone tells you the shape of the network but not *where customers live* — you need the prefix list riding along so Dijkstra's next-hop-toward-a-switch can be turned into next-hop-toward-a-prefix.
- 3. Loop-free flooding.** Each LSP carries a per-origin sequence number. A switch accepts and re-floods an LSP only if its sequence number is strictly greater than what it already has for that origin, and never re-floods back out the port the LSP arrived on (split horizon on the flood path). This — not a visited-node list — is what makes flooding terminate.
- 4. Route computation.** Every 50 ms tick, I rebuild the full routing table with Dijkstra from the current link-state database, rather than only recomputing on a detected change. At 5–15 switches a full run is a few hundred relaxation steps — negligible against the 4,000,000-instruction budget — and unconditional recompute removes an entire class of bug where a topology change silently fails to trigger a recompute. I diff the new prefix→port mapping against what's actually installed and only emit the Install/Delete actions needed to close the gap, so an unchanged result costs zero actions.
- 5. Failure recovery.** When a neighbor times out, I delete every installed route whose egress port is that neighbor's port and do *not* try to pre-install a backup path. The dead link is also dropped from my own next LSP, which reconverges the rest of the network. The next customer packet toward an affected destination re-punts as NoRoute; by then the control plane has usually already reconverged onto the new path. This is slower than a precomputed backup, but far less state and code, and at this world size the gap fits comfortably inside the recovery budget.
- 6. Prefix model.** I assume every customer prefix is a fixed /24, matching what the practice worlds hand out. This is a documented assumption, not something the program derives — it would break on a world with different addressing. A /32-per-host-observed scheme is the assumption-free alternative, noted as future work.

Tradeoffs I made on purpose

Decision	Chosen	Why / cost accepted
Route recompute timing	Every timer tick, unconditionally	Simplicity & correctness over CPU; budget headroom is huge at this scale
Dijkstra tie-break	Settle lowest switch id first on ties	Deterministic across switches given the same LSDB — reduces (not eliminates) transient micro-loops
Dead-neighbor response	Delete route, let it re-punt	Much less code than a precomputed backup path; brief unreachability window is acceptable here
Prefix length	Fixed /24	Matches practice worlds; simplest correct option, not general

Known limitation, seen in testing

Because each switch's Dijkstra recompute runs on its own independent 50 ms timer, two adjacent switches can briefly disagree about who is closer to a destination in the tens-of-ms window right after a failure, before both have re-run SPF against the post-failure topology. This shows up as a small number of packets (a few dozen out of tens of thousands, at worst) bouncing between two switches for a handful of hops before dying to TTL. It resolves itself within one tick and never caused a report-card failure in testing — the simulator explicitly does not fail A1 on this (C3 loops are informational; capacity effects are graded in A2).

Validation

Ran all 5 Part 1 (no-failure) practice worlds and all 15 Part 2 (failure) world/schedule combinations.

- Part 1 (no failures), all 5 topologies: C1–C4 all PASS, 100.00% delivery.
- Part 2 (with failures), all 15 combinations: OVERALL PASS on every run. C1 delivery 99.0–99.4% (floor 98%); C2 reachability 100% of app pairs in every run; C4 recovery 12/12 events inside the 1000 ms budget on every run (worst observed: 235 ms).

Constants: HELLO every 50 ms · neighbor timeout 150 ms · LSP refresh 200 ms · LSDB entry max age 1 s · link cost 1 (uniform) · prefix length /24.